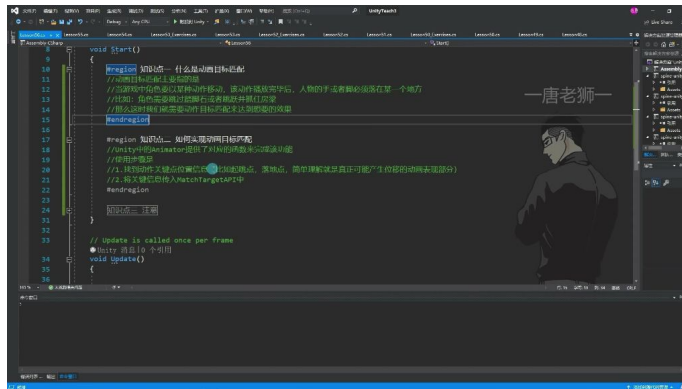


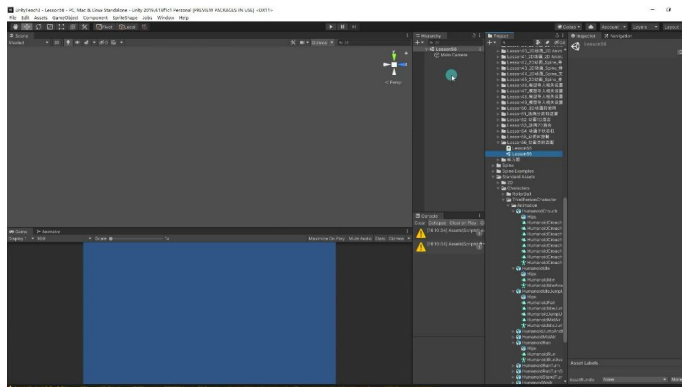
2. 如何实现动画目标匹配 03:35

1) 知识点一：什么是动画目标匹配 03:40

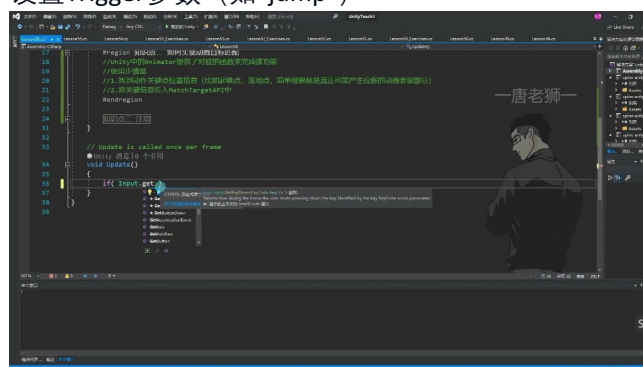


- 定义：动画目标匹配指的是当游戏中角色以特定动作移动时，动作播放完毕后人物的手或脚必须准确落在指定位置的技术。
- 应用场景：
 - 角色跳过踏板石
 - 角色跳跃并抓住房梁
- 实现原理：通过Unity的Animator组件提供的MatchTarget函数实现动作与目标位置的精确匹配。

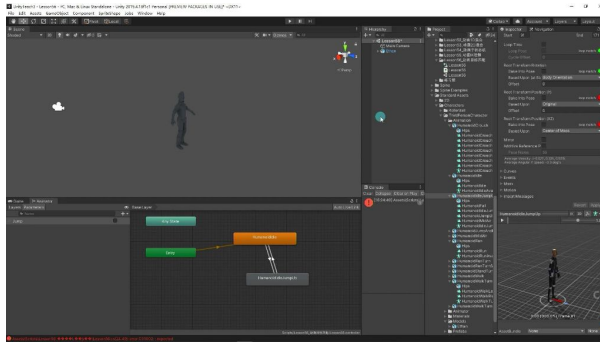
2) 知识点二：如何实现动画目标匹配 05:52



- 准备阶段：
 - 创建动画状态机
 - 添加待机动作和跳跃动作
 - 设置Trigger参数（如"jump"）
- 核心步骤：
 - 获取关键点信息：
 - 确定动作关键帧位置（起跳点、落地点等）
 - 计算动作位移百分比（如40.8%开始位移，63.9%结束位移）
 - 调用MatchTarget API：
- 参数详解：

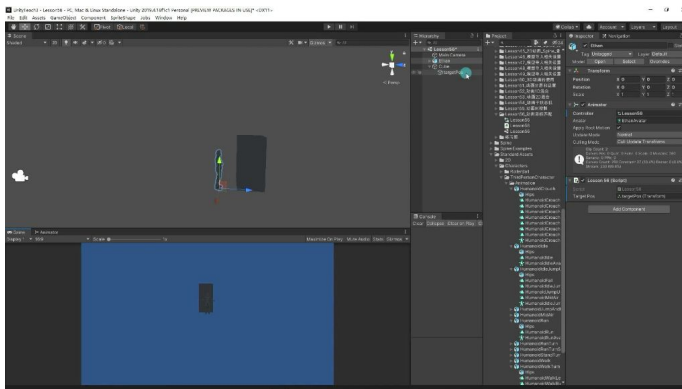


- **目标位置：**场景中目标点的Transform位置
- **目标角度：**目标点的旋转角度
- **匹配骨骼：**指定用身体哪个部位匹配（如AvatarTarget.LeftFoot）
- **权重设置：**
 - **位置权重：** Vector3.one表示XYZ轴都参与匹配
 - **旋转权重：** 1表示完全匹配旋转
- **时间范围：**
 - **开始百分比：** 0.4f（40%处开始匹配）
 - **结束百分比：** 0.64f（64%处完成匹配）



- **注意事项：**
 - 必须提前创建好动画状态机和相关参数
 - 精确计算动作的位移时间范围是关键
 - 目标点建议使用空物体标记位置
 - 测试时可能出现匹配不准确的情况，需要调整百分比参数

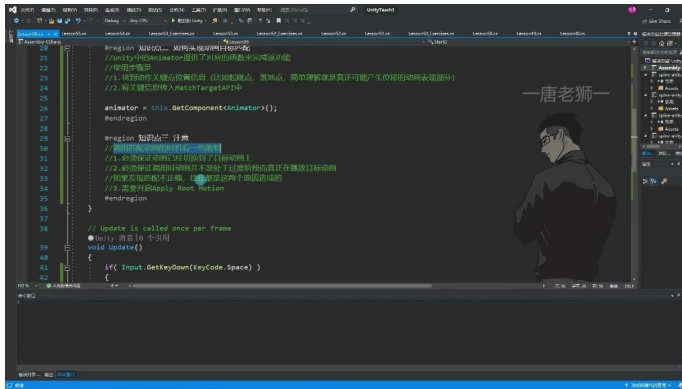
3) 知识点三：常见问题与解决方案



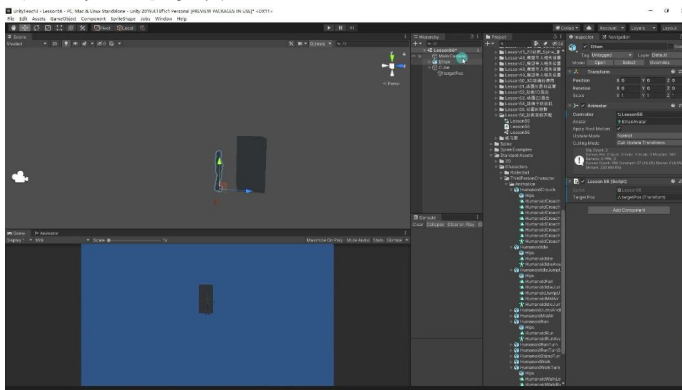
- **典型问题：**
 - 角色瞬移到目标点
 - 匹配时机不准确
 - 动作完成后位置偏移
- **解决方案：**
 - 仔细检查动作曲线，确认准确的开始和结束百分比
 - 调整MatchTargetWeightMask的权重参数
 - 确保目标点的旋转与动作需求一致
 - 测试不同骨骼点的匹配效果（如换用RightFoot或Root）

3. 动画目标匹配实现方法

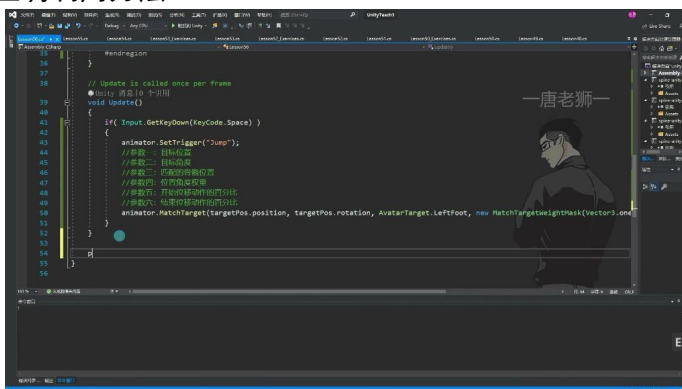
1) 基本实现步骤



- - **API功能：**Unity中的Animator提供了MatchTarget函数来实现动画目标匹配
 - **关键步骤：**
 - 找到动作关键点位置信息（如起跳点、落地点等真正产生位移的动画表现部分）
 - 将关键信息传入MatchTarget API中
 - **代码示例：**
- 2) 调用时机注意事项



- - **必要条件：**
 - 必须保证动画已经切换到了目标动画上
 - 必须保证调用时动画不处于过渡阶段，而是真正在播放目标动画
 - 需要开启Apply Root Motion参数
 - **常见问题：**匹配不正确往往是由于前两个条件未满足造成的
- 3) 正确调用方法

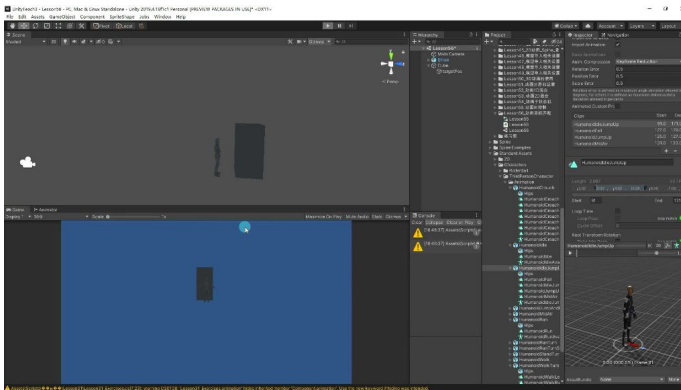


-
- **问题分析：**直接在按键触发时调用匹配函数会导致：
 - 动画可能还未切换到目标动画
 - 可能仍处于过渡阶段
- **解决方案：**
 - 在跳跃动画中添加事件（如命名为"MatchTarget"）
 - 在动画播放到合适时间点（如0.4秒前）触发事件

- 在事件回调函数中执行匹配操作
- **优势：** 确保匹配函数在动画正确播放阶段被调用，避免异常

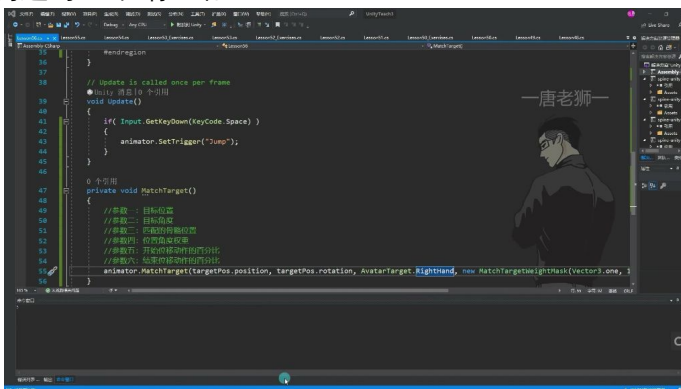
4. 应用案例 16:39

1) 例题:跳跃动画目标匹配



- **实现方法：**
 - 通过Animator.MatchTarget()函数实现动画目标匹配
 - 关键参数包括：目标位置、目标角度、匹配骨骼位置、位置角度权重、开始和结束位移动作百分比
- **调用时机：**
 - 必须保证动画已切换到目标动画
 - 必须保证调用时动画处于播放状态而非过渡阶段
 - 需要开启Apply Root Motion选项
- **调试技巧：**
 - 将匹配时机设置为跳跃动画播放到40%时效果最佳
 - 位移量不宜过大，否则会出现不自然的动作表现

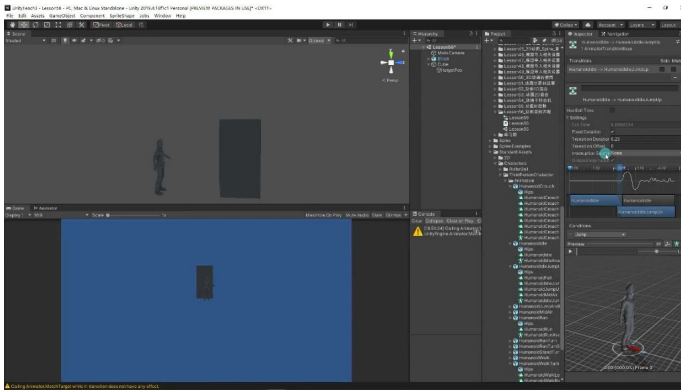
2) 例题:手上目标匹配 18:01



- **骨骼选择：**
 - 可匹配不同骨骼部位（如右手或右脚）
 - 通过AvatarTarget.RightHand/RightFoot参数指定
- **实际应用：**
 - 适用于攀爬场景（如手抓房梁）
 - 同时匹配位置和角度可实现更自然的交互效果
- **代码实现：**

```
1 animator.MatchTarget(targetPos.position, targetPos.rotation,
2     AvatarTarget.RightHand,
3     new MatchTargetWeightMask(Vector3.one, 0f),
4     0.4f, 0.6f);
```

3) 例题:事件提前问题 18:58



-
- **常见错误：**
 - 在动画过渡阶段（默认0.25秒）调用匹配会导致失效
 - 事件触发过早（如放在动画第一帧）会产生匹配异常
- **解决方案：**
 - 确保匹配调用在动画实际播放阶段
 - 通过Animator窗口检查过渡时间线
 - 调整事件触发点到过渡完成后的时间位置
- **调试验证：**
 - 将匹配点从40%调整到更早位置会重现问题
 - 正确设置后即使修改目标位置也能准确匹配

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
动画目标匹配概念	角色动作播放完毕后，身体部位必须准确落在指定位置的功能（如跳跃踏脚石、抓房梁）	与普通动画位移的区别： 美术预设位移 vs 程序动态匹配	★★
实现步骤	1. 识别动画关键点位置信息；2. 调用 MatchTarget API 传入目标位置、骨骼节点等参数	关键参数记忆点： - AvatarTarget选择 (LeftFoot/RightHand等) ； - 位移百分比范围（40%-64%）	★★★★
API核心参数	1. 目标位置/旋转；2. 匹配骨骼（枚举值）；3. 位置/角度权重 (Vector3.one) ；4. 动作起止百分比	易错点： - 角度权重需单独设置； - 位置权重需 Vector3类型	★★★
调用时机限制	1. 必须确保动画已切换完成；2. 不在过渡阶段调用；3. 需开启 ApplyRootMotion	典型错误： - 在过渡阶段调用导致瞬移； - 事件触发过早（需>40%）	★★★★
实际应用案例	1. 踏脚石连续跳跃；2. 房梁抓取动作；3. 不同高度平台跳跃	资源优化要点： 一套动画 +匹配算法替代多套美术资源	★★

